A Fool with a Tool is Still a Fool

"A fool with a tool is still a fool." — Grady Booch (attributed)[1]

If judgment is the craft that's left, what does it catch that the tool can't?

My first serious attempt with an AI coding tool was extracting an API package out of a large open source SDK. The goal was clean enough: separate the API from the implementation, two packages instead of one. Standard refactoring work, the kind of thing that should be mechanical once you know what you want.

I burned two days on it. Two days of reshuffling and redefining, no useful progress, the agent confidently helping me go in circles, producing output that looked plausible and went nowhere. I learned a lot. I produced nothing useful.

A week later I tried again. Same task, same tool. It worked.

The difference was that I now understood the system well enough to tell it what I actually needed. I'd failed my way into the right level of specification. I knew what to point out, what to constrain, what to leave open. The prompt was better because my thinking was better.

That's the thing that gets lost in the enthusiasm about AI coding tools. The tool doesn't make the engineer smarter. It makes the engineer faster, but only at the level of thinking they already have. If the engineer's understanding is shallow, it produces shallow output quickly. If the specification is vague, it fills the vagueness with confident guesses.

A fool with a tool is still a fool.

I got better over time. Prompts that would have taken me two days of burned tokens eventually took two hours. The tool started clicking once I understood what it needed to be told, what level of detail it required, where it would go wrong without guidance.

[1] Widely attributed to Grady Booch; the exact origin is unverified.

But here's what I noticed about that learning: it was entirely private.

I failed, observed, adjusted, retried. Nobody watched that process. Nobody else on the team went through it with me. I was building the understanding of how to specify, how to constrain, how to prompt for the kind of system we actually needed, and it lived in my head. It didn't transfer. It didn't accumulate anywhere the team could access it.

Which means everyone else who picked up the tool was starting from scratch. Their own two days of burned tokens. Their own private iteration loop. Learning the same lessons in isolation, making the same early mistakes, building the same understanding that never quite made it out of their head either.

AI's quietest effect on teams is the gradual privatization of operational knowledge. How to work with the agent well is hard-won understanding. And by default it goes nowhere.

I was getting faster. Tests were thorough. Implementation looked structurally clean. I had developed a review habit. Before handing anything to the team, I'd check my test cases, look for edge cases I might have missed, flag anything that seemed odd in what was being tested.

What I wasn't checking in the same way was the implementation's behavior. The agent wrote it, it passed the tests, it looked structurally sound. My attention had shifted, upward toward architecture and coverage, away from granular behavioral correctness at the line level.

There was a junior engineer on the team. First or second year. Exceptional eye for detail. He started finding things. Regularly, in a pattern, nothing alarming, just consistent. Edge cases. Behavioral inconsistencies. Small gaps between what the code did and what the project specification said it should do.

Nothing catastrophic. But in a feature flag backend, behavioral correctness isn't optional. A flag that evaluates differently than the specification says it should is exactly the kind of bug that's invisible until it isn't.

He was catching things I'd stopped looking for. Not because he was better than me. Because he was still looking at the thing my attention had drifted away from. He read for behavior. I'd started reading for structure, because structure was what the agent handed back to me and what the tests confirmed.

The tool hadn't made me worse. It had reorganized where my expertise got applied. And the reorganization created a blind spot I couldn't see, because I was focused on the output the tool produced, not on what the output wasn't telling me.

That blind spot only became visible because someone else on the team was still looking at the full picture.

What "a fool with a tool is still a fool" actually means in practice isn't that AI makes engineers foolish. Most engineers using these tools are experienced and careful, and they develop real skill with the tool over time. The problem is subtler: the tool quietly shifts what the engineer pays attention to, and the things it shifts them away from don't announce their absence. They go unnoticed until someone else catches them, or until nobody does.

The team is the check on what the tool can't see.

It works as a different set of eyes at a different level of the system, not as a safety net. The junior engineer wasn't valuable because he caught my mistakes. He was valuable because he was still reading the code the way you read code before you've developed the habit of trusting the agent's output. He was closer to the behavior than I was.

But that check only works if the team is actually looking. And the same forces that push engineers toward private AI tools push them away from collective attention. Everyone heads down with their own context, their own loop, their own hard-won understanding of how to prompt well. The team starts to feel like a set of individuals who share a codebase rather than a group of people who share a system.

The agent only validates against the prompt. If a prompt reflects one person's understanding of what the system should do, that's all it gives back. The team is the only mechanism that puts more than one person's understanding into the spec before the agent runs.

Prompting skill is the smaller part of what matters here. System thinking is the bigger one, and it's what gets built collectively: in the room, in the review, in the moment where someone with a different level of attention says that's not quite right, here's what the spec actually says.

A fair objection here: I just hadn't learned to use the tool yet. The two days of burned tokens were a skill problem, not a fundamental limitation of AI. And that's partly true. I did get better. The tool did get more useful as I understood what it needed. But that's exactly the point. The skill the tool requires didn't come with the tool, because it's the kind of thing that used to develop in the room, and when everyone learns to prompt privately, it develops alone or not at all.

Extreme Programming[2] made this argument in the nineties: understanding develops through collaboration, not in isolation. The industry mostly ignored it in favor of individual productivity metrics and frameworks that optimized the team away.

[2]Kent Beck, *Extreme Programming Explained: Embrace Change* — discussed, with citation, in the introduction.

AI is making the XP argument unavoidable by forcing the team to do what the tool can't.

The junior was doing *recognition work*: catching the wrong shape before it compounds. It's a skill, and it doesn't only belong to juniors. An eval checks output against intent someone already encoded; recognition catches drift from intent nobody had encoded yet, before there's a test to write.

It's quiet, constant work. It surfaces in a code review, in exploring an unfamiliar part of the codebase, in the planning pass before a spec goes to the agent. An engineer reaching for a pattern the team abandoned two years ago. A hand-rolled utility that duplicates what the shared library already provides. None of it is catastrophic. All of it gets more expensive the longer it stays. And none of it is something the agent catches reliably, because it doesn't know what clean looks like in this particular system, accumulated over time. The engineer does, from having been in the system, from having seen the messy version enough times to recognize it on sight. The eye develops through exposure. It can't be specced, and an agent can't be prompted to have it.

What I do with the recognition matters as much as the recognition. Reviewing agent-generated code on the JoustMania codebase, I noticed it had built three separate structures to track one piece of state where a single one would have been clean. It worked. It passed the tests. And it meant every future change would touch three places instead of one.

I didn't fix it. I created a ticket.

Not because the fix was hard. Because fixing it in place would have conflated two things, the work I was doing and the problem I'd noticed, and muddied both. Noticing without immediately fixing, naming a problem without solving it in the wrong moment, is its own discipline. AI makes it harder to practice, because the agent wants to keep going and the flow state feels productive. The pause to write the ticket instead of fixing in place is a small act of system thinking over immediate output.